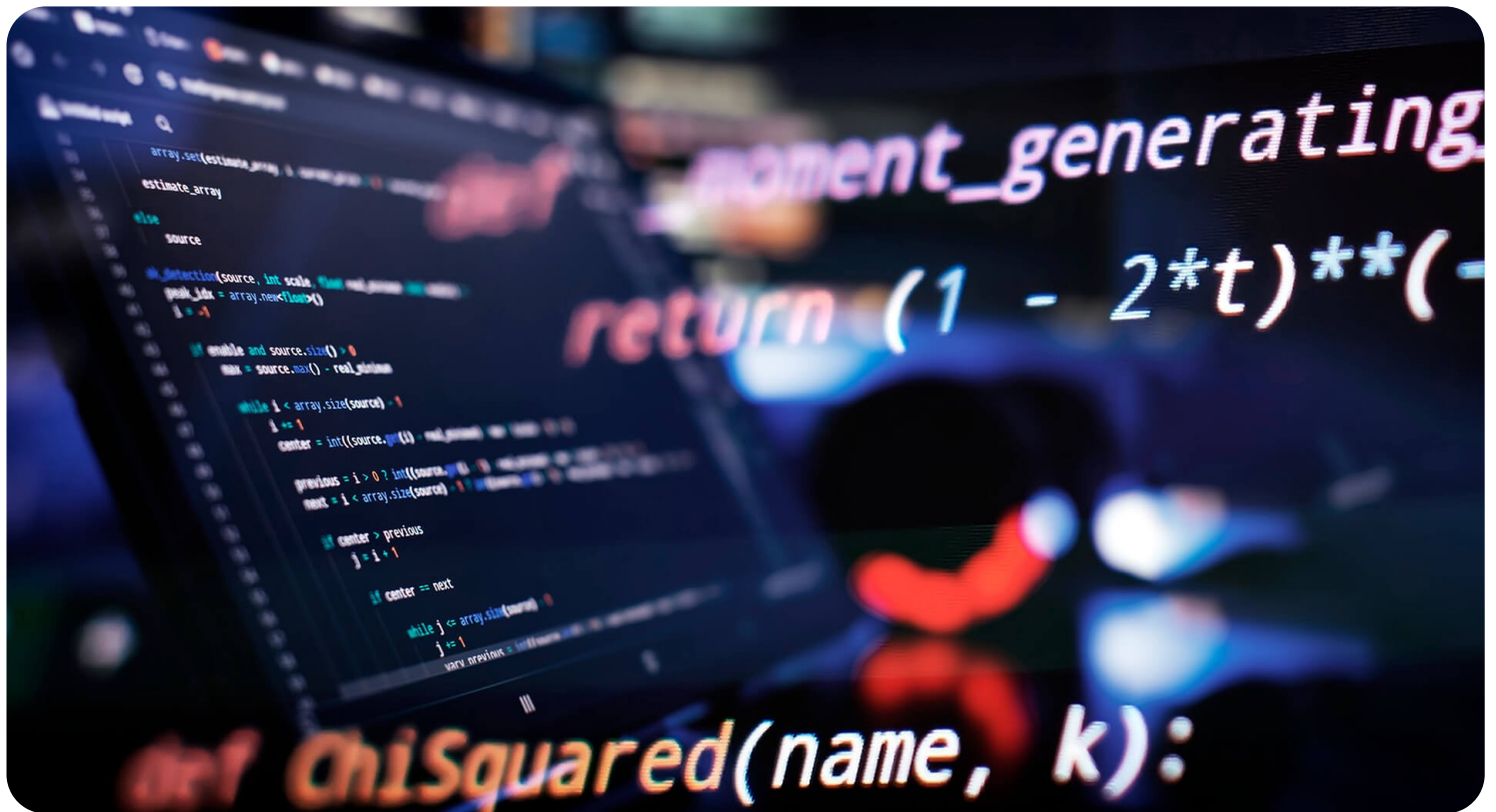




Guía de aprendizaje Documentando Microservicios con OpenAPI

Exp 3 – Semana 7

Desarrollo Backend II (PBY2202)

Facilitador disciplinar: Ányelo Castellón Ríos

Asesor par: Ignacio Villarroel Sánchez

Índice

Introducción a la semana.....	4
Resultado de aprendizaje.....	5
Conceptos relevantes.....	5
Preguntas activadoras.....	6
Actividad.....	6
Introducción a la documentación de microservicios.....	7
Importancia de la documentación estandarizada.....	7
Beneficios de Open API Specification (OAS).....	9
¿Qué es Open API Specification?.....	9
Fundamentos de Open API Specification (OAS).....	12
Elementos clave de OAS.....	12
Definición de endpoints y operaciones.....	15
Estándares y formatos de representación.....	17
Estructura y documentación de la arquitectura de microservicios.....	18
Visión general de la arquitectura.....	18
Definición de módulos y servicios.....	19
Cómo representar la arquitectura en OAS.....	21
Asegurando la consistencia y alineación con estándares.....	23
Patrones de nomenclatura y versionamiento.....	23
Buenas prácticas de estandarización.....	26
Validación y verificación de la documentación.....	28
Herramientas de edición y automatización con OAS.....	30
Editores y entornos de desarrollo.....	30

Integración en flujos de trabajo.....	32
Generación automática de documentación.....	33
Guía práctica para la elaboración de la documentación.....	34
Preparando requerimientos y lineamientos.....	34
Creación de la estructura OAS.....	36
Ejemplo práctico y ejercicios de documentación.....	39
Paso 1: Abrir el proyecto base y verificar dependencias.....	39
Paso 2: Revisar el controlador de ejemplo.....	41
Paso 3: Ejecutar la aplicación y visualizar la documentación.....	42
Paso 4: Ejercicio para afianzar conocimientos.....	45
Cierre de la semana.....	46
Referencias.....	47
Glosario.....	48
Apuntes.....	51

Introducción a la semana



En esta semana, exploraremos a profundidad la documentación de la arquitectura y los endpoints de microservicios utilizando Open API Specification (OAS). Aprenderás a estructurar y representar visualmente la arquitectura de un sistema basado en microservicios, asegurándote de que cumpla con los requerimientos específicos

del proyecto y mantenga estándares de calidad y consistencia.

Además, te enfocarás en la elaboración de documentación detallada de endpoints, que incluye la definición de métodos HTTP, parámetros, respuestas y códigos de estado, siguiendo las mejores prácticas de la industria. A través de ejemplos prácticos y actividades guiadas, desarrollarás las habilidades necesarias para crear documentación profesional y clara que facilite la comprensión y el mantenimiento de los microservicios.

Los contenidos clave que encontrarás en esta guía son:

- Introducción a Open API Specification (OAS).
- Componentes principales de la documentación OAS.
- Representación visual de la arquitectura de microservicios.
- Definición y estandarización de endpoints.
- Validación y revisión de documentación.

Resultado de aprendizaje

El estudiante será capaz de:

RA3. Documenta componentes de microservicios, usando estándares para la definición de arquitectura y endpoints a elaborar, de acuerdo a requerimientos específicos.

Indicador de logro:

IL9. Elabora la documentación de la arquitectura y los endpoints de los microservicios utilizando Open API Specification (OAS), asegurando la consistencia y alineación con los estándares.

Conceptos relevantes

Microservicios	Open API Specification (OAS)	Arquitectura
Endpoints	Métodos HTTP	Documentación
Consistencia	Validación	Herramientas

Preguntas activadoras

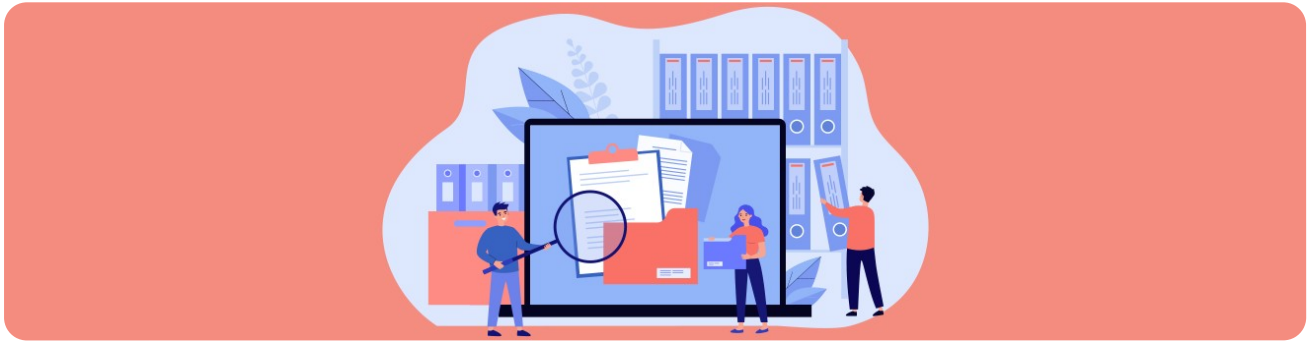
- ¿De qué manera la documentación detallada de la arquitectura de microservicios facilita la colaboración entre equipos y mejora la escalabilidad del sistema?
- ¿Cómo influye la consistencia en la documentación de endpoints en la comprensión y mantenimiento de los microservicios por parte de desarrolladores y stakeholders?
- ¿De qué forma el uso de estándares como Open API Specification (OAS) asegura la calidad y uniformidad en la definición de los endpoints?
- ¿Cómo integrarías herramientas como Swagger o Postman en tu flujo de trabajo para validar y mantener la documentación de los endpoints actualizada y alineada con los requerimientos del proyecto?

Actividad

En esta séptima semana, realizarás una actividad formativa grupal (2 a 3 integrantes), llamada "Documentando Microservicios con OpenAPI", donde deberás elaborar la documentación de la arquitectura y los endpoints de los microservicios utilizando OpenAPI Specification (OAS), asegurando la consistencia y alineación con los estándares establecidos.

Introducción a la documentación de microservicios

Importancia de la documentación estandarizada



Documentar microservicios no se reduce solo a describir sus endpoints, requiere un **enfoque estructurado** que facilite la comprensión y el mantenimiento de todo el sistema a lo largo del tiempo.

¿Por qué es crucial documentar en entornos de microservicios?

1. Complejidad incremental:

- En un sistema de microservicios, el número de servicios y sus interconexiones pueden crecer exponencialmente.
- Sin documentación, el tiempo de incorporación de nuevos desarrolladores aumenta y la corrección de errores se dificulta.

2. Coordinación de equipos descentralizados:

- Los microservicios suelen ser desarrollados por equipos diferentes, a veces geográficamente dispersos.
- Una documentación centralizada y estandarizada promueve la colaboración y la coherencia en la entrega de funcionalidades.

3. Facilita escalabilidad y mantenimiento:

- La documentación clara minimiza confusiones al escalar la aplicación.
- Permite a los equipos identificar rápidamente puntos de falla o cuellos de botella.

Elementos clave de una buena documentación

- **Estructura coherente:** se recomienda mantener un formato homogéneo (por ejemplo, a través de OAS).
- **Lenguaje claro:** expresiones directas y concisas, evitando tecnicismos innecesarios.
- **Actualización constante:** la documentación debe reflejar siempre el estado real del sistema.
- **Recursos visuales:** diagramas de arquitectura, tablas de endpoints y ejemplos de request/response.

La siguiente tabla ejemplifica cómo documentar de forma resumida la relación entre endpoints, métodos y los requerimientos de autenticación.

Tabla 1

Tabla descriptiva de Endpoints

Servicio	Endpoint	Método	Descripción	Autenticación
Servicio X	/api/v1/users	GET	Obtiene la lista de usuarios	Token JWT
Servicio X	/api/v1/users	POST	Crea un nuevo usuario	Token JWT
Servicio Y	/api/v1/payments	GET	Consulta los pagos realizados	API Key
Servicio Y	/api/v1/payments	POST	Registra un nuevo pago	API Key

Beneficios de Open API Specification (OAS)

¿Qué es Open API Specification?



La Open API Specification (OAS) es un estándar ampliamente aceptado para describir y consumir servicios REST.

Es un formato que describe de manera unificada todos los aspectos de una API REST: rutas, métodos, parámetros, códigos de respuesta, etc.

Formatos más comunes:

- **JSON:** Adecuado si se busca automatizar o integrar con otras herramientas.
- **YAML:** Más legible para humanos.

JSON

```
01 {  
02   "employees":  
03     [  
04       {  
05         "id": "40159",  
06         "name": "John",  
07         "technology": "Cloud computing",  
08         "title": "Engineer",  
09         "team": "Development"  
10       }  
11     ]  
12 }  
13
```

YAML

```
01  
02 employees:  
03   - id: '40159'  
04     name: John  
05     technology: Cloud computing  
06     title: Engineer  
07     team: Development  
08  
09  
10  
11  
12  
13
```

Veamos un ejemplo en YAML:

Figura 1

Ejemplo simplificado en YAML

```
openapi: 3.0.3
info:
  title: API de Ejemplo
  version: 1.0.0
paths:
  /users:
    get:
      summary: Obtener usuarios
      responses:
        '200':
          description: Lista de usuarios
```

Nota. Ejemplo de especificación OpenAPI para un endpoint que devuelve una lista de usuarios. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Ventajas de usar OAS

1. Estandarización y consistencia:

- Todos los servicios se documentan de forma homogénea.
- Facilita la colaboración y la revisión de la documentación por diferentes equipos.
- Generación automática de portales y clientes:
- Herramientas como Swagger o Redoc permiten generar portales de documentación interactiva y SDKs para distintos lenguajes a partir del archivo OAS.

2. Integración con CI/CD:

- Es posible validar y generar la documentación de manera automatizada con cada despliegue, garantizando que siempre se mantenga actualizada.

Link de interés

En el siguiente enlace puedes revisar toda la información existente de OpenAPI Specification, a través de Swagger:

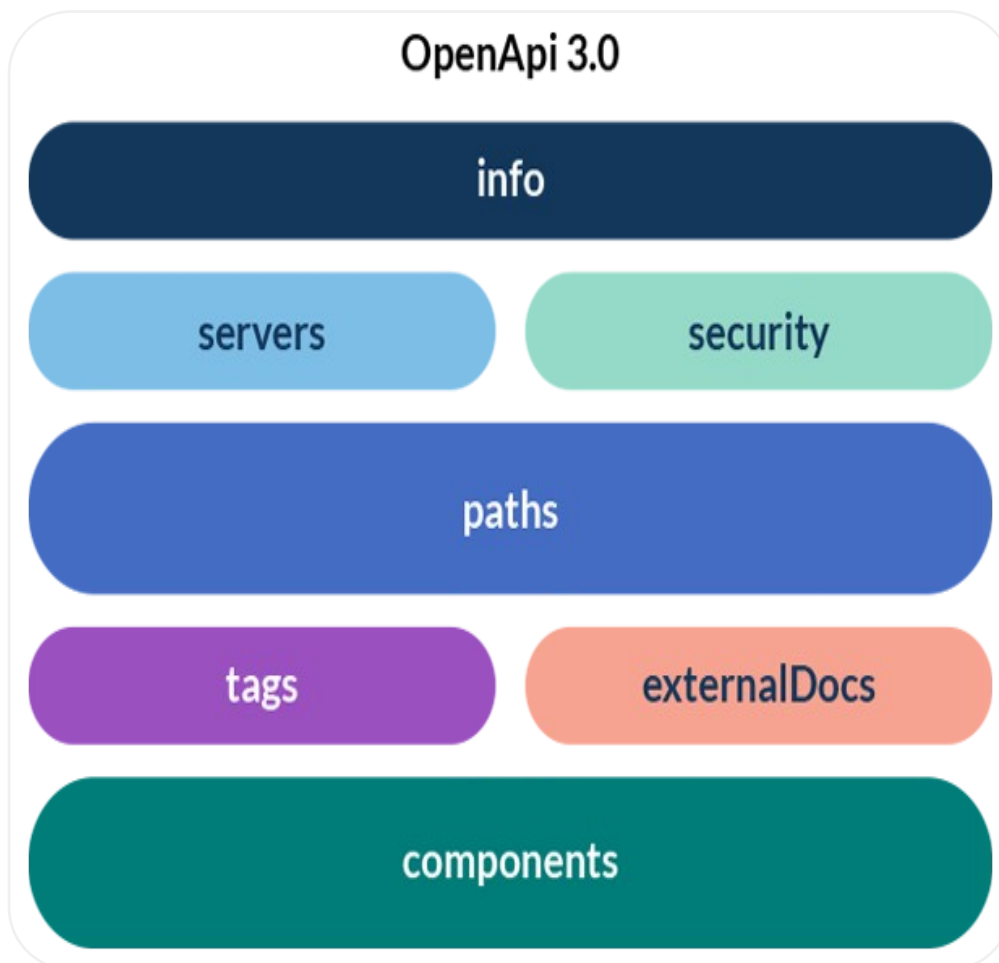
<https://swagger.io/specification/>

Fundamentos de Open API Specification (OAS)

Open API Specification es ampliamente utilizada en el mundo del desarrollo de microservicios, ya que brinda una visión clara de rutas, métodos, parámetros y respuestas, facilitando la colaboración entre equipos y la integración de sistemas.

Elementos clave de OAS

En esta etapa es importante familiarizarte con los términos clave que aparecen en la especificación, pues a partir de ellos podrás crear un documento ordenado y fácil de mantener.



1. Info

- Indica la información de la API, como título, versión, descripción y términos de licencia.
- Facilita la identificación y versión de la API cuando trabajas con diversos servicios.

2. Servers

- Lista las URL base en las que se expone la API (producción, desarrollo, pruebas, etc.).
- Permite indicar el entorno al que se apunta según la configuración.

3. Security

- Describe los esquemas de autenticación y autorización que se requieren para consumir la API.
- Puede incluir mecanismos como API keys, tokens JWT o flujos OAuth2.

4. Paths

- Define las rutas disponibles de la API.
- Cada ruta describe los métodos HTTP y la información necesaria para su funcionamiento.

5. Tags

- Organiza las rutas de la API en categorías lógicas.
- Facilita la navegación y segmentación de la documentación según el contexto.

6. ExternalDocs

- Proporciona enlaces externos a documentación adicional.
- Expande la información más allá del alcance principal de la documentación de la API.

7. Components

- Agrupa definiciones reutilizables, como schemas (modelos de datos), parámetros comunes y estructuras de respuesta.
- Ayuda a mantener la coherencia en la definición de objetos que se repiten en distintos endpoints.

A continuación, se muestra una tabla con los elementos principales que suele contener un documento OAS:

Tabla 2

Elementos principales que contiene un documento OAS

Componente	Función principal	Ejemplo en YAML/JSON
info	Metadatos de la API (título, versión, descripción)	title: "API de Inventario"
servers	URLs base de despliegue	https://api.ejemplo.com/v1, localhost
security	Esquemas de autenticación	BearerAuth, ApiKeyAuth
paths	Rutas y métodos HTTP	/productos, /usuarios, /ordenes
Tags	Categorías lógicas para agrupar rutas	tags: [{name: "Productos"}, {name: "Usuarios"}]
externalDocs	Documentación externa relacionada	externalDocs: {description: "Más detalles", url: "https://docs.ejemplo.com"}
components	Definiciones reutilizables (schemas, responses)	UserSchema, ErrorResponse

Definición de endpoints y operaciones

Endpoints y operaciones constituyen el corazón de la API, pues aquí se detallan las funcionalidades disponibles para el consumo y la interacción con el sistema.

1. Rutas (paths)

- Se definen bajo la sección paths del documento OAS.
- Cada ruta corresponde a un recurso o acción (por ejemplo: /usuarios, /productos/{id}, etc.).

2. Métodos HTTP

- Se especifican dentro de cada path: GET, POST, PUT, DELETE, PATCH, entre otros.
- Incluyen un summary o description para aclarar su propósito.
- Permiten detallar parámetros de entrada, cuerpos de petición (request body) y códigos de estado en las respuestas (responses).

3. Parámetros y datos de entrada

- Pueden ubicarse en path (por ejemplo, {id}), en query (por ejemplo, ? nombre=Juan), en headers o en el cuerpo de la solicitud.
- Facilitan la configuración dinámica de cada operación.

4. Respuestas y códigos de estado

- Se describen en la sección responses.
- Utilizan códigos de estado HTTP (200, 201, 404, 500, etc.) e incluyen descripciones claras.
- Es común adicionar ejemplos de respuesta para guiar a los consumidores de la API.

Aquí tienes un ejemplo en formato YAML para una ruta de productos:

Figura 2

Ejemplo de documentación de Endpoint de Producto

```
paths:
  /productos/{id}:
    get:
      summary: Obtener información de un producto
      description: Retorna los datos completos del producto especificado por su ID.
      parameters:
        - name: id
          in: path
          required: true
          description: Identificador único del producto
          schema:
            type: integer
      responses:
        '200':
          description: Datos del producto retornados correctamente
        '404':
          description: Producto no encontrado
```

Nota. Especificación OpenAPI para obtener información detallada de un producto mediante su ID. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Estándares y formatos de representación

Mantener un estándar uniforme te permitirá reducir confusiones y facilitar la integración con herramientas que procesan o validan las definiciones de la API.

1. Formato YAML vs JSON

- YAML se considera más legible y adecuado para la edición manual.
- JSON facilita la integración con sistemas de automatización o scripts que consumen las definiciones.

2. Consistencia en la nomenclatura

- Nombrar endpoints y campos de forma clara y descriptiva.
- Utilizar convenciones coherentes para variables como `id_producto` o `productId` según la decisión de tu equipo.

3. Lineamientos recomendados

- Usar la última versión de la especificación OpenAPI (generalmente 3.x).
- Separar definiciones de esquemas en la sección components para evitar duplicaciones.
- Actualizar la documentación con cada cambio en los endpoints o funcionalidades de la API.

A modo de ejemplo, puedes ver un diagrama que ilustra el flujo típico para definir y mantener la documentación de tu API:

Figura 3

Flujo de trabajo para documentar APIs



Nota. Proceso para crear, validar, publicar y mantener actualizada la documentación de una API.

Estructura y documentación de la arquitectura de microservicios

Es importante recordar que, al trabajar con microservicios, cada servicio debe ser fácilmente independiente, pero, a la vez, capaz de comunicarse y compartir datos con el resto de componentes en la arquitectura. Mantener un registro coherente y actualizado de estos servicios es fundamental para el éxito del proyecto.

Visión general de la arquitectura

Entender la visión global ayuda a que todos los integrantes del equipo sepan de qué manera se relacionan los componentes, qué dependencias tienen y cuáles son los puntos críticos que se deben supervisar.

1. Identificación de servicios

- Se deben listar y nombrar los microservicios de forma clara.
- Es recomendable describir su función general: servicio de pagos, servicio de usuarios, servicio de notificaciones, entre otros.

2. Patrón de comunicación

- Los microservicios pueden comunicarse de forma síncrona (REST, gRPC) o asíncrona (mensajería, colas).
- Determinar el patrón correcto influye directamente en la escalabilidad y robustez del sistema.

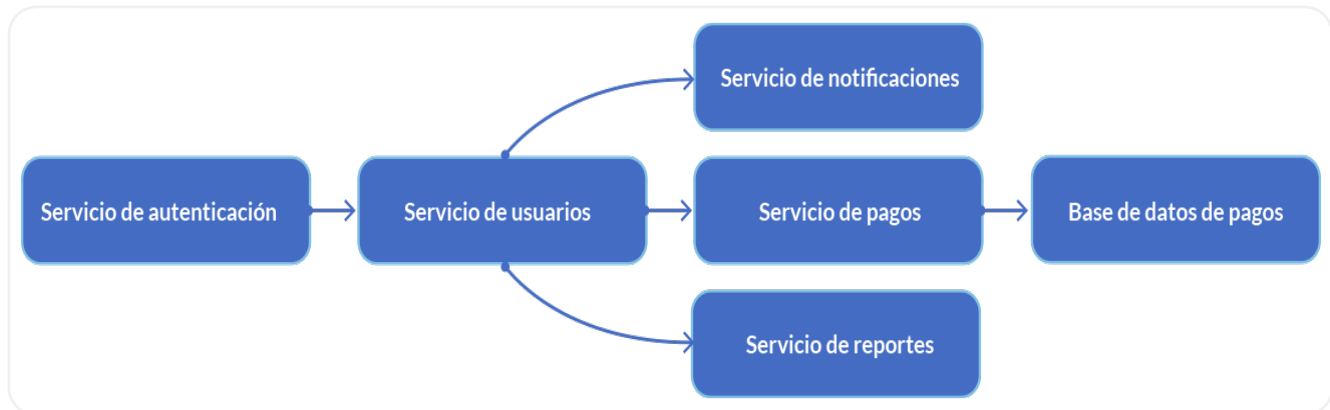
3. Visión de dependencias

- Un mapa de dependencias aclara quién usa o consume la información de otro servicio.
- Este gráfico reduce el riesgo de introducir cambios en un servicio que afecten negativamente a todo el ecosistema.

A continuación, se presenta un **diagrama conceptual** que ilustra de manera genérica cómo podría verse la disposición de varios microservicios:

Figura 4

Diagrama de Arquitectura de Microservicios4



Nota. Representación de servicios interconectados como autenticación, usuarios, pagos, notificaciones y reportes.

En este ejemplo, se aprecia cómo el Servicio de autenticación otorga tokens al Servicio de usuarios, que a su vez interactúa con notificaciones, pagos y reportes. Cada flecha indica un punto de comunicación que posteriormente se documentará con mayor detalle.

Definición de módulos y servicios

La definición clara de los módulos ayuda a cada equipo o desarrollador a entender qué hace cada pieza, qué datos maneja y en qué formato entrega o recibe la información.

1. Rol y responsabilidades

- Cada servicio debe tener un objetivo claro y bien definido.
- Explica de forma concreta qué problema o conjunto de problemas resuelve y cuáles son sus límites (bounded context).

2. Dependencias internas y externas

- Un microservicio puede usar librerías o componentes internos que no estén expuestos.
- Igualmente, puede requerir la conexión con servicios externos (servicios de terceros, API públicas).

3. Tabla de referencia de servicios

- Recopilar información en una tabla puede agilizar la consulta y el mantenimiento de la documentación.

La siguiente tabla ejemplifica la definición de cada servicio, sus principales responsabilidades y las dependencias que maneja:

Tabla 3

Definición de cada servicio, responsabilidades y dependencias

Servicio	Responsabilidad principal	Dependencias externas	Base de datos
Servicio de usuarios	Registrar, autenticar y administrar datos de usuarios	Servicio de autenticación	Base de datos de usuarios
Servicio de pagos	Procesar transacciones y manejar facturación	API de pasarelas de pago	Base de datos de pagos
Servicio de notificaciones	Enviar correos y notificaciones push/sms	Servicio de correo, SMS Gateway	NoSQL para colas y logs
Servicio de reportes	Generar informes y estadísticas de uso	Servicios de usuarios y pagos	Base de datos analítica

Cada columna se puede expandir con detalles más específicos, como versión de la API, protocolos soportados, planes de escalabilidad y métodos de autenticación requeridos.

Cómo representar la arquitectura en OAS

El estándar OAS no solo sirve para describir endpoints individuales, sino que también puede emplearse para aclarar cómo se relacionan entre sí los distintos servicios.

1. Descripción global

- OAS permite, en la sección info, dar una descripción general de la API y, opcionalmente, incluir elementos adicionales que aporten un panorama de la arquitectura.
- Se sugiere un archivo OAS para cada microservicio, manteniendo uniformidad en la estructura de la documentación.

2. Refinamiento del componente servers

- Se pueden listar varios servidores, indicando entornos de desarrollo, testing y producción.
- En arquitecturas complejas, cada microservicio tendrá su propio server, lo que puede facilitar la configuración en equipos de integración continua.

3. Uso de tags y components

- Para describir el “módulo” o la “responsabilidad” de cada endpoint, se aconseja el uso de tags.
- Se pueden crear secciones de components compartidas para representar modelos de datos que utilicen varios microservicios.

4. Enlaces y referencias cruzadas

- OpenAPI 3.x ofrece la posibilidad de crear enlaces (links) que indiquen la relación entre operaciones de un servicio y otro.
- Estos enlaces permiten a los consumidores de la API ver cómo navegar entre los distintos endpoints y servicios.

Un ejemplo básico de cómo podrías describir en OAS la relación de tu servicio con otro:

Figura 5

Especificación OpenAPI del Microservicio de Pagos

```
openapi: 3.0.3
info:
  title: "Microservicio de Pagos"
  version: "1.0"

paths:
  /transacciones/{id}:
    get:
      summary: "Obtener detalles de una transacción"
      responses:
        '200':
          description: "Detalles completos de la transacción"
          links:
            usuarioRelacionado:
              operationId: getUsuario
              parameters:
                id: "$response.body#/usuarioId"
              description: "Obtiene la información del usuario asociado a esta transacción"
```

Nota. Define el endpoint para obtener detalles de una transacción con enlaces a información relacionada. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

En este ejemplo, el link `usuarioRelacionado` apunta a otra operación (`getUsuario`) que se describe posiblemente en el Microservicio de usuarios, reflejando la relación que existe entre la entidad transacción y el usuario que la generó.

Asegurando la consistencia y alineación con estándares



Mantener un lenguaje uniforme y una estructura clara en la documentación es esencial para facilitar la lectura, el mantenimiento y el trabajo colaborativo. A continuación, encontrarás recomendaciones prácticas y recursos que te ayudarán a lograrlo.

Patrones de nomenclatura y versionamiento

Este punto describe cómo establecer convenciones de nombres para endpoints, versiones de APIs y recursos, fomentando la claridad y la comunicación entre equipos.

1. Convenciones de nombres para endpoints

- Identifica un estilo consistente, por ejemplo, usar nombres en plural (e.g., /users, /posts).
- Incluye versiones de la API en el endpoint (e.g., /v1/users) para indicar cambios relevantes.

2. Versionamiento de la API

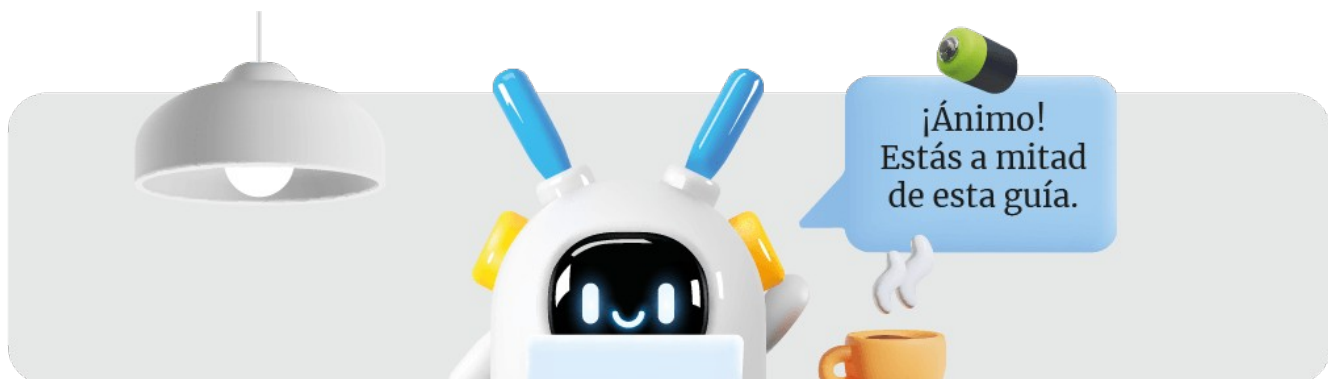
- Emplea un enfoque SemVer (Semantic Versioning) que utiliza el formato MAJOR.MINOR.PATCH (por ejemplo, 1.0.0), donde cada número indica la magnitud del cambio.
- Mantén claridad en cuándo se incrementa cada parte del número de versión para reflejar nuevos añadidos, correcciones y cambios disruptivos.

Tabla 4

Pautas de nomenclatura

Aspecto	Ejemplo	Justificación
Endpoints	/v1/products	Indica versión y recurso en plural
Recursos anidados	/v1/users/{id}/orders	Muestra relación jerárquica entre entidades
Parámetros	?status=active	Define filtros o paginación de forma clara
Versionamiento	MAJOR.MINOR.PATCH	Facilita la identificación de rupturas y nuevas funcionalidades

Siempre es recomendable que cada cambio en la API sea planeado y documentado, de modo que los consumidores del servicio puedan adaptarse sin generar conflictos.



¿Cuál sería la forma?

Supongamos que se está diseñando una API para una tienda en línea. Esta es una posible estructura de endpoints con sus versiones:

Versión 1.0.0

1. Obtener productos:
 - GET /v1/products
 - Retorna todos los productos con soporte para parámetros como filtros (?category=electronics) y paginación (?page=1).
2. Obtener órdenes de un usuario:
 - GET /v1/users/{id}/orders
 - Muestra todas las órdenes relacionadas con un usuario específico identificado por {id}.
3. Actualizar el estado de un pedido:
 - PATCH /v1/orders/{id}
 - Permite modificar atributos de un pedido, como su estado (status).

Versión 2.0.0

En esta nueva versión, se introduce un cambio disruptivo:

1. Cambios en el endpoint de productos:
 - El recurso ahora incluye subcategorías.
 - Nuevo endpoint: GET /v2/products?subcategory=laptops.
2. Eliminación de un endpoint anterior:
 - Se elimina /v1/orders y se reemplaza con un sistema de eventos para manejo de pedidos en tiempo real.

Buenas prácticas de estandarización

A continuación, te mostraremos la relevancia de seguir lineamientos comunes para facilitar la comprensión y el mantenimiento de la documentación y el código.

- **Uniformidad en la escritura**
 - Adopta un lenguaje coherente y profesional en cada parte de la documentación.
 - Evita la mezcla de idiomas o tecnicismos excesivos que obstaculicen la lectura.
- **Uso de un estilo homogéneo**
 - Define un formato para la descripción de endpoints: objetivo, parámetros, ejemplos de request/response.
 - Asegura que cada integrante del equipo aplique el mismo estilo.
- **Plantillas y guías internas**
 - Crea formatos o plantillas para describir nuevas funcionalidades o endpoints.
 - Ofrece lineamientos claros y ejemplos para que todos los miembros del equipo los sigan.

Ejemplo de formato sugerido para documentar un endpoint

Tabla 5

Formato sugerido para documentar un endpoint

Sección	Descripción
Nombre	Nombre del endpoint, por ejemplo: "Obtener detalle de producto"
Método y ruta	GET /v1/products/{id}
Objetivo	Retorna la información detallada de un producto según su identificador
Parámetros	id: Identificador del producto (path param).
Códigos de estado	200 (OK), 404 (No encontrado), 500 (Error interno)
Ejemplos	Request: GET /v1/products/123
	Response (200): { "id": 123, "name": "X" }

El uso de tablas y formatos uniformes facilita la búsqueda de información y reduce la probabilidad de errores.

Validación y verificación de la documentación

En este punto se mencionan herramientas y métodos de control de calidad para asegurar la exactitud de la documentación, garantizando que la información se mantenga al día y refleje la realidad del proyecto.

	 Swagger Editor Supported by SMARTBEAR	 Spectral
Herramientas de validación automática	Permite revisar la sintaxis de un archivo Open API Specification y señalar errores o inconsistencias.	Ofrece reglas personalizables para validar que la documentación cumpla con convenciones y estándares establecidos.
Pruebas de smoke para la API	Ejecuta pruebas rápidas que confirmen que los endpoints documentados responden con los códigos de estado esperados.	Verifica que la estructura de los modelos de datos coincida con la definición en la documentación.
Revisiones cruzadas y code reviews	Involucra a múltiples integrantes del equipo para revisar la documentación y comparar las definiciones con el comportamiento real de la API.	Registra las observaciones en un repositorio o sistema de control de versiones para mantener un historial de ajustes.

Figura 6

Proceso de Actualización y Validación de APIs



Nota. Flujo para actualizar endpoints, validar especificaciones OAS y realizar pruebas de API.

Este flujo resume cómo se puede integrar la verificación documental en el ciclo de desarrollo, maximizando la coherencia y la calidad de la API.

Herramientas de edición y automatización con OAS

La idea principal consiste en que puedas utilizar OAS de forma eficiente a través de distintas herramientas técnicas. Mantener la documentación actualizada e integrada al proceso de desarrollo agiliza la colaboración entre equipos, reduce errores y permite que otras partes interesadas accedan rápidamente a la información de la API.

Editores y entornos de desarrollo

Muchos editores y entornos de desarrollo ofrecen soporte para OAS, lo que facilita la creación y validación de la documentación. A continuación, se listan algunas opciones populares:

Swagger Editor  Swagger Editor Supported by SMARTBEAR	Redoc  Redoc	VS Code y sus plugins 
• Permite editar y validar archivos OAS (YAML o JSON) en tiempo real. • Ofrece una vista previa de cómo lucirá la documentación generada. • Facilita la compartición de la definición con tu equipo.	• Convierte un archivo OAS en una documentación web navegable con un diseño limpio. • Brinda menús desplegables, barras de búsqueda y secciones claras. • Muy útil para exponer la documentación a colaboradores o clientes.	• Visual Studio Code cuenta con extensiones específicas para trabajar con OAS. • Resaltan la sintaxis, validan el documento y ofrecen autocompletado. • Integran el trabajo de documentación directamente en el IDE.

Tabla 6*Herramientas recomendadas*

Herramienta	Características principales	Disponibilidad
Swagger Editor	Edición y validación en tiempo real	Versión web y local (mediante Docker)
Redoc	Documentación web interactiva	Integrable con npm o scripts standalone
VS Code plugins	Autocompletado, resaltado de sintaxis, linting de YAML/JSON	Extensiones en Marketplace

El uso de estas herramientas te permitirá acelerar la redacción de la documentación y, al mismo tiempo, garantizar que cumpla con la especificación OAS.

Integración en flujos de trabajo

Insertar la generación y validación de la documentación en el ciclo de vida del proyecto ayuda a mantener la coherencia entre lo que se desarrolla y lo que se publica. Un ejemplo de flujo de trabajo podría verse así:

1. Repositorio central con el archivo OAS

- Almacenar el archivo OAS en la misma ubicación del código que describe.
- Hacer cambios en la documentación cada vez que se modifican endpoints o recursos.

2. Validación continua

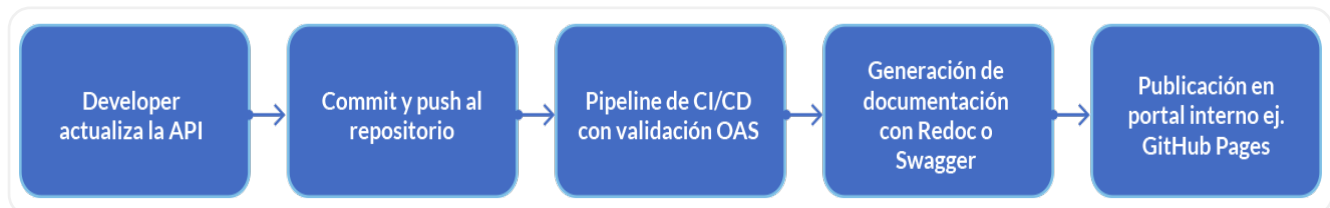
- Incluir pasos de verificación en el pipeline de CI/CD que revisen la sintaxis o sigan reglas personalizadas (por ejemplo, Spectral).
- Evitar que se fusionen o desplieguen ramas con definiciones rotas o desactualizadas.

3. Publicación automatizada

- Generar la documentación de manera automática en cada commit o pull request.
- Publicarla en un portal interno o servicio externo, de modo que toda la organización acceda a la versión más reciente.

Figura 7

Flujo de trabajo de actualización de API



Nota. Diagrama que muestra el proceso desde que un desarrollador actualiza una API hasta que la documentación actualizada se publica en un portal interno.

Integrar estos pasos reduce la posibilidad de que la documentación se quede desactualizada y ayuda a los equipos a saber siempre qué versión de la API está en producción.

Generación automática de documentación

La generación automática de documentación aporta ventajas cuando se busca minimizar esfuerzos manuales y mantener sincronizados el código y la definición del API. A continuación, se proponen algunos enfoques:

1. Extracción desde anotaciones

- Varios frameworks permiten anotar el código con metadatos que describen la API.
- Herramientas como springdoc-openapi (en Java Spring) generan el archivo OAS basándose en esas anotaciones.

2. Scripts de unificación

- Si el proyecto consta de múltiples microservicios, cada uno podría tener su propio archivo OAS.
- Existen utilidades como swagger-cli para fusionar definiciones en un solo documento, lo que ayuda a presentar una visión global.

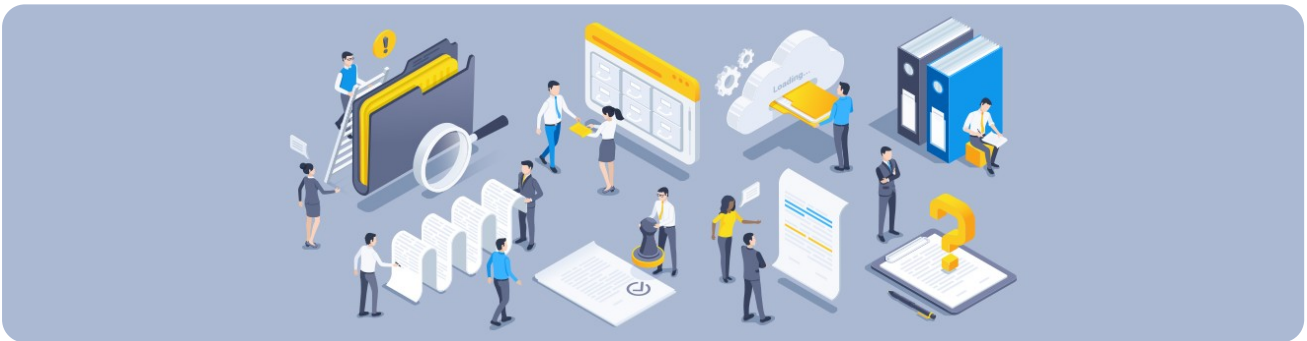
3. Librerías y ejemplos

- springdoc-openapi (Java/Spring): Transforma anotaciones en controladores en definiciones OAS.
- Swashbuckle (C#/.NET): Genera un portal Swagger directamente desde tus controladores.
- Sphinxcontrib-openapi (Python): Combina documentación tradicional con la de OAS.

Cada solución se adapta a contextos y lenguajes diferentes, por lo que conviene evaluar qué herramienta se ajusta mejor a la arquitectura y requerimientos de tu proyecto.

Guía práctica para la elaboración de la documentación

En esta sección, nos adentraremos en el proceso de documentar la arquitectura y los endpoints de tus microservicios. Aprenderás a aplicar los principios de una buena documentación para crear un recurso valioso para ti, tu equipo y los consumidores de tus APIs.



Documentar adecuadamente tus microservicios facilita su comprensión, mantenimiento y evolución. Una buena documentación permite a los desarrolladores entender cómo funciona cada servicio, cómo interactuar con él y cómo integrarlo con otros sistemas.

Preparando requerimientos y lineamientos

Antes de comenzar a documentar, es crucial reunir la información esencial y establecer lineamientos claros. Esto asegura que la documentación sea completa, precisa y coherente.

Define los objetivos de la documentación

Comienza por determinar qué se espera lograr con la documentación. ¿Buscas facilitar la integración de los microservicios? ¿Deseas proporcionar una guía para el equipo de desarrollo? ¿O quieres crear una referencia para los usuarios finales de la API?

Definir los objetivos te ayudará a enfocar la documentación y a seleccionar la información más relevante.

Recopila información clave

Reúne toda la información relevante sobre los microservicios, incluyendo:

- Nombre del microservicio: un nombre descriptivo que identifique claramente la función del servicio.
- Versión del microservicio: indica la versión actual del servicio, lo que facilita el seguimiento de los cambios y la gestión de compatibilidad.
- Descripción funcional: describe de forma concisa la funcionalidad principal del microservicio.
- Dependencias: lista las dependencias del microservicio, tanto internas como externas. Esto incluye otros microservicios, bases de datos, bibliotecas, etc.
- Tecnologías utilizadas: especifica las tecnologías empleadas para desarrollar el microservicio, como lenguajes de programación, frameworks, etc.
- Endpoints: describe cada endpoint del microservicio, incluyendo su propósito, método HTTP, parámetros de entrada y salida, y códigos de respuesta.

Establece lineamientos de estilo

Define un conjunto de lineamientos de estilo para asegurar la consistencia y la legibilidad de la documentación. Esto puede incluir:

- Formato: elige un formato para la documentación, como Markdown, HTML o una herramienta específica de documentación de API.
- Lenguaje: define el lenguaje a utilizar en la documentación.
- Tono: establece un tono consistente para la documentación, ya sea formal o informal.
- Estructura: define una estructura clara para la documentación, con secciones y subsecciones bien organizadas.

Creación de la estructura OAS

OpenAPI Specification (OAS) es un estándar para describir APIs RESTful. Utilizar OAS te permite definir la estructura de tus APIs de forma clara y legible, tanto para humanos como para máquinas.

Inicia un archivo OAS

Un archivo OAS se define en formato YAML o JSON. Puedes comenzar con un archivo básico que incluya la información general de la API:

Figura 8

Iniciando con OpenAPI

```
openapi: 3.0.0
info:
  title: Nombre de la API
  version: v1
```

Nota. Fragmento de especificación OpenAPI 3.0.0, definiendo el título y la versión de la API.

JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Define la sección paths

La sección paths describe los endpoints de tu API. Cada endpoint se define con su método HTTP (GET, POST, PUT, DELETE, etc.) y una descripción de su funcionalidad.

Figura 9

Definiendo un Endpoint

```
paths:
  /usuarios:
    get:
      summary: Obtiene una lista de usuarios.
      responses:
        '200':
          description: Lista de usuarios.
```

Nota. Especificación del endpoint /usuarios con método GET para obtener una lista de usuarios, usando OpenAPI. Código de respuesta 200. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Define la sección components

La sección components te permite definir elementos reutilizables en tu API, como esquemas, parámetros y respuestas.

Figura 10

Modelando datos

```
components:
  schemas:
    Usuario:
      type: object
      properties:
        id:
          type: integer
        nombre:
          type: string
```

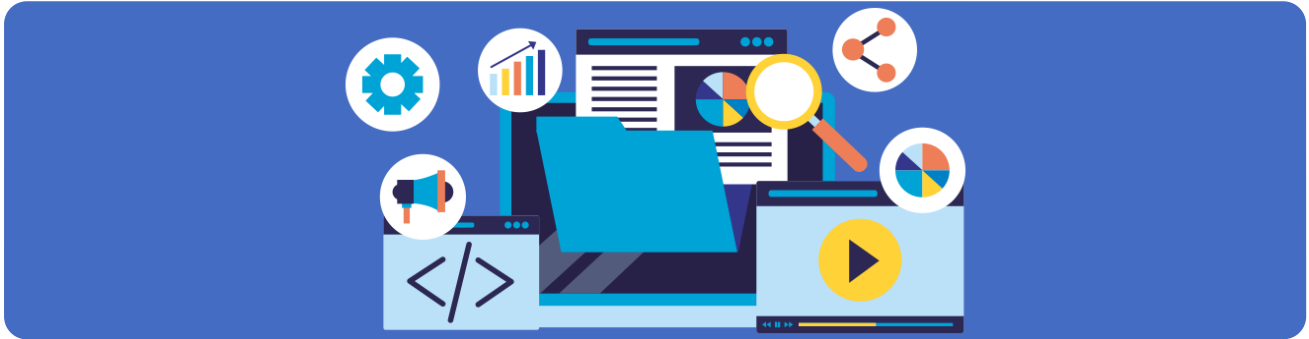
Nota. Definición del esquema Usuario dentro de la sección components de OpenAPI.

Describe un objeto con propiedades id (entero) y nombre (string). JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Asegura la consistencia

Mantén la consistencia en la estructura y el estilo de tu archivo OAS. Utiliza nombres descriptivos para los endpoints y parámetros, y documenta claramente cada elemento.

Ejemplo práctico y ejercicios de documentación



A continuación, encontrarás un ejemplo práctico de cómo generar documentación para tu API mediante Spring Boot y springdoc-openapi. La idea es que, con los pasos descritos, seas capaz de ver cómo se configuran los controladores y la integración de la librería para documentar automáticamente tus endpoints. No necesitarás descargar ni generar un proyecto por tu cuenta, ya que se te proporcionará un esqueleto de proyecto listo para usar.

Paso 1: Abrir el proyecto base y verificar dependencias

1. Recibir el proyecto

Descargable

Se te proporcionará un proyecto Spring Boot ya configurado con Spring Web y Springdoc OpenAPI WebMVC UI en el archivo pom.xml.:

https://ava.duoc.cl/bbcswebdav/xid-5318674_1

Luego, abre el proyecto en tu IDE (Eclipse, IntelliJ, VS Code, etc.) para explorar la estructura.

2. Revisar el archivo pom.xml

Comprueba que exista la dependencia de springdoc-openapi-starter-webmvc-ui. Por ejemplo:

Figura 11

Dependencia para Springdoc OpenAPI

```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>2.1.0</version>  
</dependency>
```

Nota. Configuración de Maven para integrar documentación OpenAPI en aplicaciones Spring Boot. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Asegúrate de que el proyecto reconozca esta dependencia al sincronizar con Maven.

Paso 2: Revisar el controlador de ejemplo

1. Verificar la clase SaludoController

El proyecto base incluye un paquete controller con una clase SaludoController.java.

Ejemplo de controlador:

Figura 12

Controlador de Saludos en Spring Boot

```
@RestController
@RequestMapping("/api/saludo")
public class SaludoController {

    @GetMapping
    public ResponseEntity<String> obtenerSaludo() {
        return ResponseEntity.ok("¡Hola desde Spring Boot!");
    }

    @PostMapping
    public ResponseEntity<String> crearSaludo(@RequestBody String nombre) {
        return ResponseEntity.ok("¡Hola, " + nombre + "! Tu saludo ha sido creado."
    }
}
```

Nota. Define endpoints para obtener y crear saludos mediante métodos GET y POST.

JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

2. Entender los endpoints

- GET /api/saludo: retorna un mensaje de saludo.
- POST /api/saludo: recibe un nombre y retorna un mensaje personalizado.
-

¿Qué logras con este paso?

Identificas la lógica básica de los endpoints y confirmas que el controlador cumpla su función de exponer rutas REST.

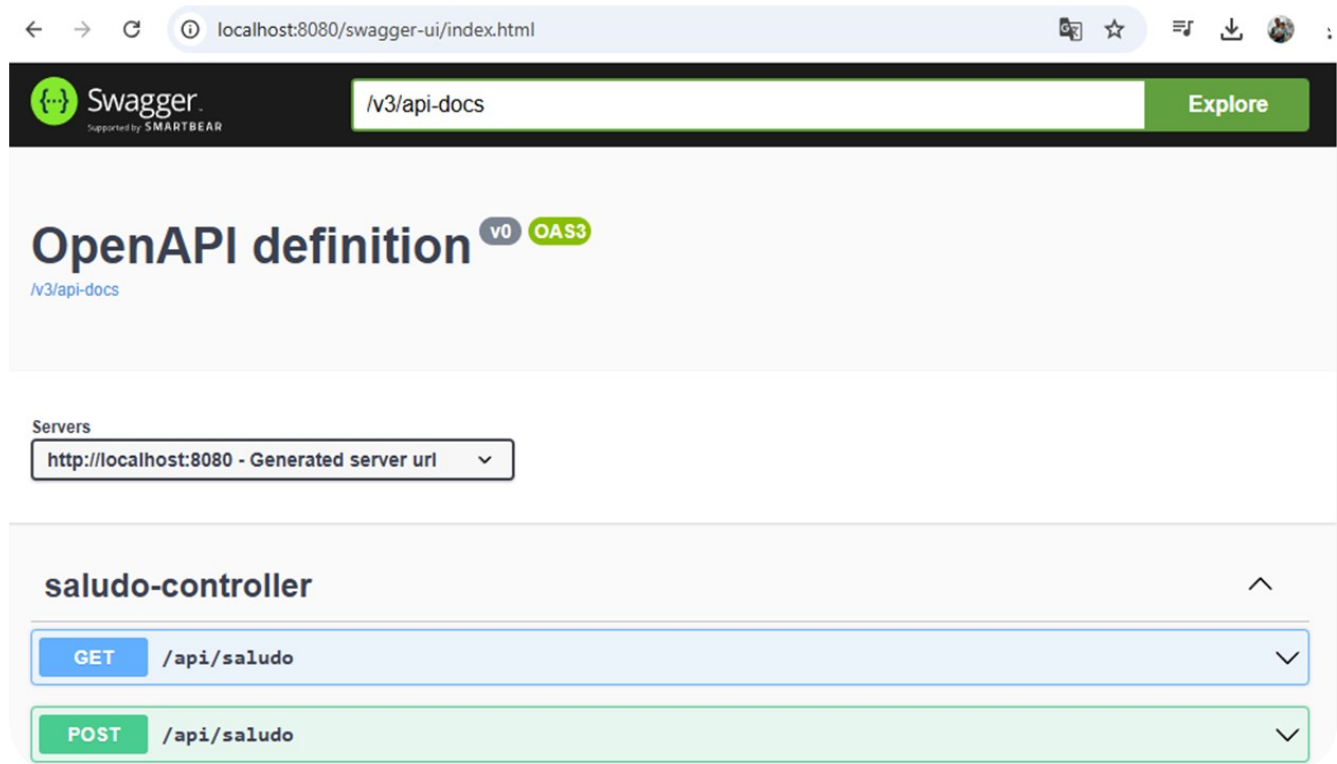
Paso 3: Ejecutar la aplicación y visualizar la documentación

1. Correr la aplicación

Ejecuta la clase principal (por ejemplo, DemoOpenapiApplication) y observa la consola para asegurarte de que Spring Boot se inicie correctamente.

2. Abrir Swagger UI

- Visita <http://localhost:8080/swagger-ui.html> (el puerto puede variar según la configuración).
- Deberías ver una interfaz que lista los endpoints del proyecto.
- Podrás probarlos directamente desde la interfaz o simplemente leer sus definiciones.

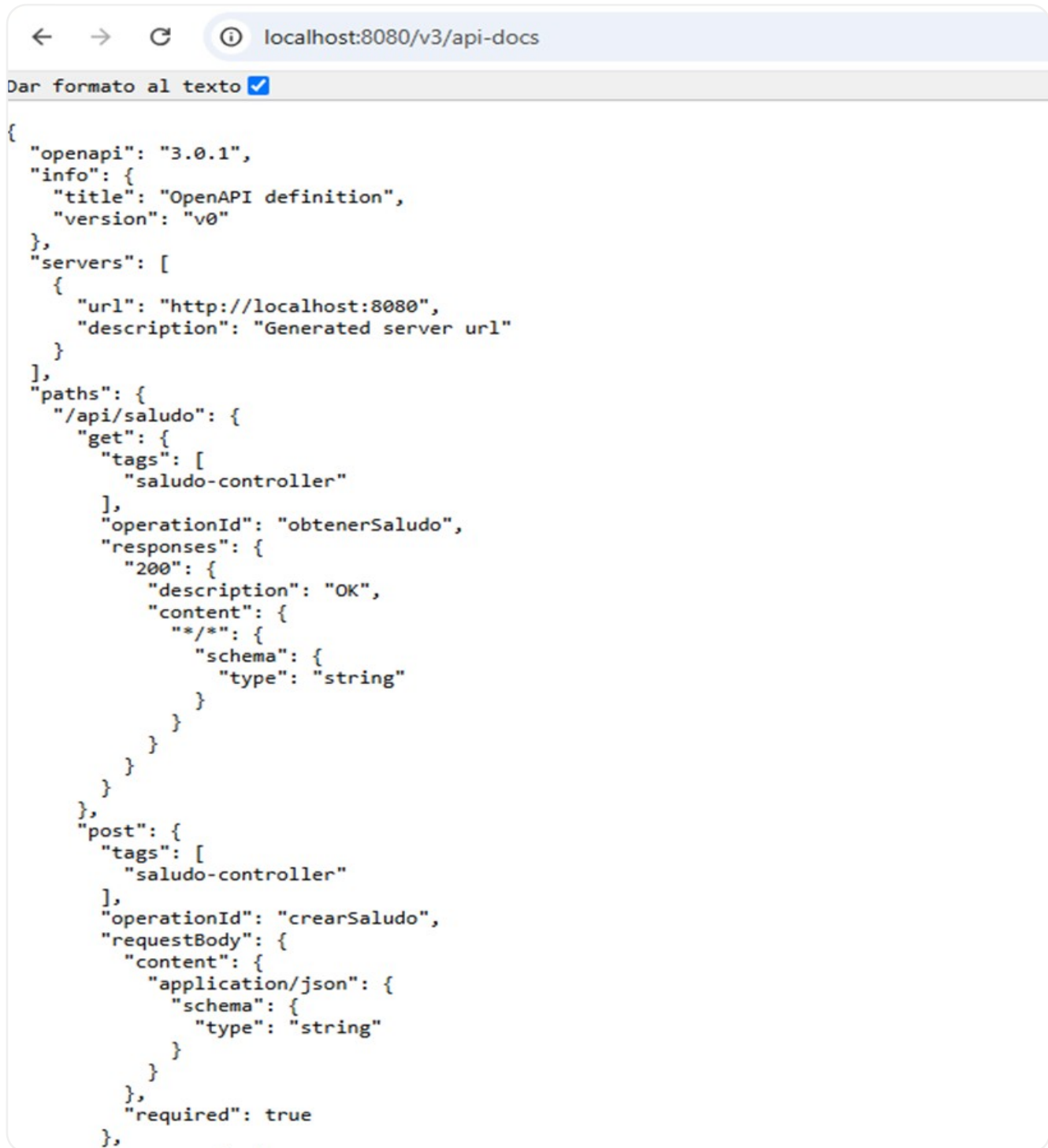
Figura 13*Exploración de API con Swagger UI*

Nota. Visualización de los endpoints definidos en el controlador de saludos. Smartbear. (2024). Swagger. (OAS 3.0). <https://swagger.io/>

3. Consultar la definición OpenAPI

- Si deseas ver el archivo JSON generado automáticamente, dirígete a <http://localhost:8080/v3/api-docs>.
- Este archivo describe por completo tu API, incluyendo métodos, parámetros y códigos de respuesta.

Figura 14

Documento OpenAPI Generado

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "paths": {
    "/api/saludo": {
      "get": {
        "tags": [
          "saludo-controller"
        ],
        "operationId": "obtenerSaludo",
        "responses": {
          "200": {
            "description": "OK",
            "content": {
              "*/*": {
                "schema": {
                  "type": "string"
                }
              }
            }
          }
        }
      },
      "post": {
        "tags": [
          "saludo-controller"
        ],
        "operationId": "crearSaludo",
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "type": "string"
              }
            }
          }
        },
        "required": true
      }
    }
  }
}
```

Nota. Especificación JSON de la API para los endpoints definidos en el controlador de saludos.

¿Qué logras con este paso?

Constatas que la documentación automática está funcionando: cada endpoint declarado en tu controlador aparece en la interfaz, brindando detalles sobre su uso.

Paso 4: Ejercicio para afianzar conocimientos

1. Crear un segundo controlador

Por ejemplo, MascotaController, con métodos GET para listar y POST para crear, simulando un registro de mascotas.

2. Anotar cada método

Incluye `@Operation`, `@ApiResponses` e incluso `@Parameter` para describir parámetros específicos.

3. Verificar en Swagger UI

Asegúrate de que tu nuevo controlador y endpoints aparezcan correctamente documentados y sean navegables.

4. Opcional: Combinar con un cliente

Si lo deseas, genera un cliente en otro lenguaje (Python, TypeScript, etc.) usando el JSON en <http://localhost:8080/v3/api-docs> mediante OpenAPI Generator (<https://openapi-generator.tech/>).

¿Qué logras con este paso?

Te familiarizas con la extensión del proyecto, añadiendo más endpoints y validando la actualización automática de la documentación.

Cierre de la semana

A lo largo de esta semana, hemos aprendido a utilizar y aprovechar diversas herramientas de edición y automatización con Open API Specification (OAS), centrándonos en cómo integrarlas de forma eficiente en el ciclo de vida del desarrollo de microservicios. Empezamos con la selección de editores y entornos de desarrollo que simplifican la creación de la documentación, para después sumergirnos en la

incorporación de estos recursos en flujos de trabajo CI/CD, de manera que la definición de la API se mantenga siempre actualizada y confiable. Por último, identificamos técnicas y librerías que permiten generar documentación automática y unificar aportes de diferentes servicios, lo que promueve la coherencia y reduce esfuerzos de mantenimiento. Con estos conocimientos, la documentación de las APIs cobra una mayor solidez y se convierte en un recurso dinámico, alineado a la evolución continua del proyecto.



Referencias

- Dependabot[bot]. (2024). *Stoplightio / Spectral GitHub Repository*. <https://github.com/stoplightio/spectral>
- Smartbear. (s.f.). *Swagger Editor Documentation*. <https://swagger.io/docs/open-source-tools/swagger-editor/>
- TheLinuxFoundation. (s.f.). *OpenAPI Initiative*. <https://www.openapis.org/>
- Varchuk, A. (2024). *Redoc Documentation*. <https://github.com/Redocly/redoc>

Glosario

En el siguiente glosario encontrarás términos esenciales en la seguridad de aplicaciones backend y el uso OAS. Te invitamos a leerlos y familiarizarte con todos ellos.

Término	Definición
Microservicios	Arquitectura de software donde las aplicaciones se dividen en servicios independientes que se comunican entre sí, facilitando la escalabilidad.
API	Interfaz de Programación de Aplicaciones, conjunto de métodos y definiciones que permiten la comunicación entre distintos componentes de software.
Endpoint	Punto específico de acceso a un servicio o recurso en una API, generalmente representado como una ruta HTTP.
OAS (OpenAPI)	Especificación que define un estándar para describir APIs REST, brindando estructura y claridad en la documentación de endpoints y modelos de datos.
YAML	Formato de serialización de datos que usa indentación para delimitar estructuras, frecuentemente empleado en archivos de configuración o definición.
JSON	Formato de intercambio de datos basado en texto y estructura de objetos, muy usado para transmitir información entre cliente y servidor.
CI/CD	Integración Continua/Despliegue Continuo, metodología que automatiza la compilación, prueba y despliegue del software para

mejorar la calidad.

Refactorización Proceso de reescritura y optimización del código sin alterar su comportamiento externo, mejorando legibilidad y mantenibilidad.

Versión Semántica Esquema de numeración (MAJOR.MINOR.PATCH) para indicar mejoras y cambios en las APIs, permitiendo controlar la compatibilidad.

Documentación viva Documento que se actualiza de forma continua y automática, reflejando siempre el estado real del sistema o API en desarrollo.

Swagger Editor Herramienta en línea que permite crear, editar y validar archivos OAS, proporcionando retroalimentación instantánea.

Redoc Generador de documentación web que presenta los contenidos de un archivo OAS en un formato navegable e interactivo.

Linting Proceso de análisis de código o definiciones para verificar su estilo y consistencia, detectando errores o desviaciones de estándares.

Branch Rama de desarrollo en un sistema de control de versiones, permitiendo trabajar en paralelo sin afectar la rama principal del proyecto.

Pull Request Petición para fusionar cambios de una rama a otra, generalmente tras revisión y verificación de la calidad del código o definiciones.

Testing Conjunto de actividades destinadas a validar que el software cumpla con los requerimientos y funcione correctamente.

Pipeline Secuencia de pasos o tareas automatizadas dentro de CI/CD que se ejecutan con cada cambio de código, garantizando calidad y

consistencia.

Repositorio

Ubicación centralizada donde se almacena el código fuente, la documentación y el historial de versiones de un proyecto.

Mocking

Técnica para simular componentes de software o servicios externos, facilitando pruebas sin depender de implementaciones reales.

Consumidor de API

Aplicación o cliente que realiza peticiones a la API, consumiendo sus endpoints para obtener datos o ejecutar operaciones.

Apuntes

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.